Ontogeny: The C Compiler Lineage

How this particular family of programs developed — the direct line from Ritchie's original to the compilers in this directory.

The bootstrap problem

A compiler is a program. A C compiler is a program written in some language — possibly C itself. The question this raises is immediate: if the C compiler is written in C, how was the first C compiler compiled?

The answer is: it wasn't. Not at first.

The first C compiler was written in B — an earlier language, itself descended from BCPL, itself derived from CPL, all the way back to Algol. B ran on the PDP-7 and PDP-11 at Bell Labs. When Ritchie's C compiler could compile a useful subset of C, he rewrote it in C, compiled the new version with the old B compiler, and from that point forward the C compiler compiled itself. The B compiler was retired.

This process — writing a compiler in the language it compiles, then using an older compiler to bootstrap it into existence — is called *self-hosting*. Every compiler in this directory is self-hosted. Every compiler you use today is self-hosted.

Thompson's "Reflections on Trusting Trust" (1984) describes what this implies: the bootstrap chain goes all the way back, and if any link in the chain was ever compromised, the compromise propagates invisibly forward through every subsequent compiler. The argument is not paranoid — it is a precise description of a real property of all software compilation. It is beautiful and disturbing in equal measure.

1972 — The Original: DMR's C Compiler

Author: Dennis M. Ritchie
Platform: PDP-11
Written in: Initially B; rewritten in C once C could compile C

Approximate size: 4,000 lines
Where to find it: Unix V6 source at tuhs.org

The original C compiler is short by any standard except the one that applied in 1972, when memory was measured in kilobytes and every line had to earn its place. Ritchie wrote it while simultaneously designing the language it compiled — the two activities were inseparable. C was defined by what the compiler accepted. The language and the implementation grew together.

The architecture was direct: a single-pass compiler with no separate optimizer. Read C source. Produce PDP-11 assembly. Call the assembler. There was no intermediate representation, no passes, no separation of concerns beyond what the structure of the code imposed. The front end and the code generator were one continuous thing.

This is worth reading even if you cannot run it. The simplicity is not naivety. It is a precise match between the tool and its purpose: a compiler for a new language, on a specific machine, in a constrained environment, by a programmer who knew exactly what he needed.

Not in this directory. Available through the Unix Heritage Society at tuhs.org.

1977 — PCC: The Portable C Compiler

Author: Steve Johnson
Platform: Originally PDP-11; designed from the start for portability
Written in: C
Size: ~12,000 lines (core)
Where to find it: OpenBSD pcc project; pcc.ludd.ltu.se

By 1977, Unix was spreading to machines other than the PDP-11. Each new machine had a different instruction set, different registers, different calling conventions. Ritchie's compiler, designed for the PDP-11, needed to be ported — and each port was effectively an independent reimplementation. Someone had to write the machine code generation from scratch each time.

Steve Johnson solved this by dividing the compiler into two parts: a machine-independent front end that parsed C and produced an abstract representation, and a machine-specific back end that walked the representation and produced instructions for the target architecture. Adding a new platform meant writing a new back end, not starting over. The front end stayed fixed.

This separation — front end and back end as distinct, replaceable components — is now so fundamental that it is invisible. Every compiler you encounter today has it. LLVM built an entire

infrastructure around the idea. Johnson invented it as a practical necessity in 1977, and it turned out to be the architecture of all compilers to come.

PCC was the standard Unix C compiler for twenty years. BSD Unix shipped it. System V shipped it. AT&T used it. It was replaced by GCC in most contexts in the early 1990s, but OpenBSD maintained their own version through the 2000s as an alternative to GCC's GPL license.

Johnson also wrote yacc — Yet Another Compiler Compiler — at Bell Labs in 1970. Yacc is a parser generator: you describe your grammar, yacc produces a parser for it. It is still in use. His contribution to compiler infrastructure is larger than his name recognition suggests.

Not in this directory. Available through the OpenBSD pcc project.

1987 — GCC: The GNU C Compiler

Author: Richard M. Stallman, with decades of subsequent contributors
Platform: Originally VAX-11; now effectively everything
Written in: C (later partially C++)
Size: Millions of lines across GCC proper and its frontends
Where to find it: `gcc/` in this directory; also gnu.org/software/gcc

Richard Stallman started GCC in 1987 as the compiler for the GNU operating system — an operating system that did not yet exist, built from tools that had to be written before the system itself could exist. GCC was the flagship. Without a free C compiler, the rest of the GNU system could not be built. Without a free C compiler, any free operating system would require a proprietary tool to compile itself. That was unacceptable to Stallman, and so he wrote one.

The original GCC was a modified version of an existing compiler (the Pastel compiler). Stallman rewrote it substantially to compile C well, then extended it to C++, then Fortran, then Ada, then Java, then Go. GCC became not just a C compiler but a compiler *infrastructure* — a platform for adding new front ends (new languages) and new back ends (new target architectures) through a common framework. The infrastructure abstraction is Johnson's PCC idea, extended and generalized.

GCC introduced RTL — Register Transfer Language — as its intermediate representation. RTL sits between the parsed source and the machine code. It is close enough to machine instructions to enable real code generation, but abstract enough to allow optimization passes to modify it without touching the front end. The optimization passes run on RTL. This was a significant architectural advance: GCC can improve the intermediate form in ways that DMR's original compiler — which produced assembly

directly — could not.

GCC is also the most politically explicit codebase in this course. The GNU project has a point of view, and it is present in the design decisions, in the license (the GPL, which requires that derived works also be free), in the comments, in the documentation, and in the history of the project's governance. You cannot fully understand GCC without understanding why Stallman wrote it: not to build a better compiler, but to ensure that no programmer would ever be required to use a compiler they could not modify, study, and share. This is not packaging for the code. It is the code's reason for existing.

In this directory.

2001 — TCC: Tiny C Compiler

Author: Fabrice Bellard
Platform: x86 (originally); now x86-64, ARM, and others
Written in: C
Size: ~82,000 lines (all targets combined; the core is much smaller)
Where to find it: `tcc/` in this directory; also bellard.org/tcc

Fabrice Bellard wrote the core of TCC over a weekend in 2001 as an entry in the International Obfuscated C Code Contest — and won first prize. He then continued developing it as a real tool, but the insight from the contest entry remained: a compiler does not have to be large. A compiler does not have to be slow to build. A compiler does not have to be complex to be complete.

TCC makes explicit tradeoffs. It does not optimize aggressively. It compiles C source very fast — fast enough that the compilation step essentially disappears. GCC produces faster code; TCC produces compiled code faster. For development, for scripting contexts, for situations where the code only runs a few times, TCC's tradeoff is often the right one.

TCC has a party trick: it can execute C source files directly, as scripts. `tcc -run hello.c` compiles in memory and runs immediately. This collapses the compile step entirely for development and small tool use. It makes C behave like a scripting language while remaining entirely compiled.

The architecture of TCC is unusual and deliberately simple. Almost all of it is in a single file (`tcc.c`), with machine-specific back ends alongside. The front end, the parser, and the code generator are deeply interleaved — a deliberate inversion of the PCC and GCC philosophy of strict separation. For a program of this size, the interleaving works. You can read the whole thing. You can hold the entire design in your head. For GCC, this is impossible. For TCC, it is the point.

In this directory. Start here.

The family tree

```
DMR's C compiler (1972)
    ■  single-pass, PDP-11, no IR
    ■
    ■■■■ PCC (1977) — Johnson's front/back-end split
         ■  first portable C compiler
         ■  basis for BSD and System V compilers for 20 years
         ■
         ■■■■ GCC (1987) — Stallman's free reimplementation
              ■  multi-language, multi-target, RTL intermediate representat
ion
              ■  political: GPL, GNU project
              ■
              ■■■■ LLVM/Clang (2003–2007) — reaction against GCC complexit
y
                   modular IR (LLVM IR), BSD license
                   (not in this directory)

TCC (2001) — Bellard's minimal fresh start
    single-file, no IR, no optimizer, maximal simplicity
    (in this directory)
```

The tree is not strictly a tree in the code sense — both GCC and TCC are fresh reimplementations rather than forks of earlier source. But the intellectual lineage is real. Every node on this tree knew what came before. Every node was a response to it.

What to notice when you read TCC and GCC together

You are seeing two different answers to the same question: *what should a C compiler be?*

TCC's answer: small, fast to compile, readable end-to-end, no optimizer. Trust the programmer to write reasonably efficient code.

GCC's answer: free, complete, extensible to other languages, portable to every architecture, heavily optimizing, politically principled. Build the infrastructure that makes all future work possible.

Neither answer is wrong. The disagreement is about what matters most, and it produces genuinely different programs. This is one of the things you are learning to read: not just how a program works, but what problem its author thought they were solving.

Next: open `03-phylogeny.pdf` *— where compilers came from, how C came to exist, the geographies and characters behind all of this.*